ISSUE: Cells may contain formulas, while we want only static data to expedite cleaning and for exporting.

TRADE OFFS:

Converting from formulas to static data makes our dataset inflexible to new data being added if there is data that depends on the data within the sheet. Examples of dependent data would be running totals or datestamps, which are made dead upon conversion to static data.

Since the goal as outlined is to clean for export, and not to be used for adding further data, this may be fine. Further, the cleaning paradigm clearly outlines that copies of the sheets will be made prior to cleaning, meaning that we do not risk much by making our data static.

SOLUTION:

The following function in our cleaning script will handle converting formulas into static data:

```
function convertFormulas(sheet) {
      const range = sheet.getDataRange();
      range.copyTo(range, SpreadsheetApp.CopyPasteType.PASTE_VALUES, false);
}
```

PROCESS:

- Parameter: A tab of a spreadsheet is given to the convertFormulas function, this tab is referred to as “sheet” by the function.
- Body:
    - const range = sheet.getDataRange()
        - The function getDataRange() automatically returns information about our sheet such as the location of the first and last row with data and the location of the first and last column with data. We can think of this as a box being drawn around our data and that “range” stores the borders of this box
    - range.copyTo(range, SpreadsheetApp.CopyPasteType.PASTE_VALUES, false);
        - copyTo()
            - The function copyTo() uses the information about the borders of our data placed into “range” to identify which cells of “sheet” will be copied
        - range
            - The first argument, “range”, is the destination. Here it is the same “range” we are copying from, so the copy lands on top of the original cells

        - SpreadsheetApp.CopyPastType.PASTE_VALUES
            - The second argument tells Sheets to paste only the displayed values of the source cells. Formulas are not carried over, and formatting, notes, and data validation are left in place
        - false
            - The third argument, means the data is not transposed, so rows stay rows and columns stay columns
        - Because the values are copied directly from cell to cell, they are never read into the script and written back. Sheets does not re-interpret them, so text stats exactly as it was
        - Overwrites the formulas with the static values they display, thereby replacing formulas with static data

TESTING:

To confirm the conversion is safe, we wrote unit tests in FormulaConversionUnitTests.gs, that verify three things after conversion, these are stored within the FormulaConversionTesting.xlsx spreadsheet:

1. No formulas remain in the sheet
2. Cell formatting, how values are displayed, is unchanged
3. Each cell's data type (text, number, date, true/false) is preserved, or correctly assigned once the formula is gone

Where applicable, every test applies all three checks along with a manual visual check of the sheet. The cases covered are the following:

- Formula results
    - A single formula returning each of: text, number, true/false, date, and a blank string
    - Multiple formulas returning each of those types
- Formula results displayed with common formats such as currency, percentages, dates, decimal places do not interfere and persist
- Special formulas
    - Formulas that return errors are translated into strings describing the error
    - Formulas wrapped in error-handling functions
- A sheet with a single cell
- Formulas that reference other cells either above or below, for each result type above, as well as formulas that reference other formulas
- Formulas tied to changing data, like current date, confirming the value is frozen at the moment of conversion
- Formulas that produce more than plain values
    - Formulas that fill multiple cells at once

The cases below have not yet been thoroughly tested. We may not currently use these formulas, but they should be tested in the future to support scalability.

- Formulas that produce more than plain values
    - Formulas that create clickable links
    - Formulas that display images or in-cell charts
- External data: Formulas that pull data from outside the sheet

VALIDATION:

In order to ensure correct data handling, I also included a validation function with the purpose of checking for formulas that have not been tested. This works to avoid potential errors by keeping an acceptlist of formulas whose behavior had been tested. When a formula that is not on the accepted list is found the program will print out the following:

```
[SheetName] Unapproved FORMULANAME in cell: =FORMULANAME(...)
...
Needs manual verification: 1 formula(s) flagged across the spreadsheet
```

This allows for manual inspection in the case of an unexpected formula (such as the to be tested cases above.

The list of approved formulas are the following:

- Logical: AND, FALSE, IF, IFERROR, IFNA
- Math: SUM
- Statistical: AVERAGE
- Date: DATE, TODAY
- Operator: CONCAT, DIVIDE
- Array: ARRAYFORMULA
- Info: NA

ISSUE: Empty cells in sheets makes file size too large

DEFINITION:

Empty, the way we think of it, may not be the same as the computer considers it. We likely want to remove any data that does not have significance, this includes a much broader range than just simply empty cells. The following cases, do not provide significant data and should be remove from our sheets:

- Empty cells
- Formula cells returning “”
- Cells with whitespace
    - Non-breaking spaces
    - Zero-width characters
- Leading apostrophes

SOLUTION ONE:

1.

Secondly, we need to identify where empty cells can be located within our sheet to ensure we are targeting the ones we want to remove.

CASES:

Case 1: Empty Row
First, let’s discuss empty rows. These should, in all likelihood, always be removed. There are three positions our empty rows can be located in:

Subcase 1.1: Empty row above the valid data
Subase 1.2: Empty row centrally located within the valid data (Y-Bound)
Subase 1.3: Empty row below the valid data

Case 2: Empty Column
Second, we can have empty columns. These also may also be deleted unless the lack of entries gives significance to the data. These also can be located in three positions (left, X bounded, or right of the data) but also have the added complexity of being a headed column or not, leading to six new cases:

Subcase 2.1: Empty column left of the valid data and headed
Subcase 2.2: Empty column left of the valid data and not headed
Subcase 2.3: Empty column centrally located within the valid data and headed
Subcase 2.4: Empty column centrally located within the valid data and not headed
Subcase 2.5: Empty column right of the valid data and headed
Subcase 2.6: Empty column right of the valid data and not headed

Case 3: Internal Empty Cell
Finally, we want to discuss empty cells that are embedded within non-empty rows and non-empty columns. These must be cells that are both X and Y bounded, else they would fall into one of the above cases. There are two cases associated with these.

Subcase 3.1: Empty cell in headed column
Subcase 3.2: Empty cell in an unlabeled column

We can represent these cases on a cell level. The following two tables explain what cases might apply to an empty cell. If all the cases are handled, then no cell in that position will be empty after the cleaning:

	Headed		
	Left	X Bound	Right
Above	1.1 OR 2.1	1.1 OR 2.3	1.1 OR 2.5
Y Bound	1.2 OR 2.1	1.2 OR 2.3 OR 3.1	1.2 OR 2.5
Below	1.3 OR 2.1	1.3 OR 2.3	1.3 OR 2.5

	Not Headed		
	Left	X Bound	Right
Above	1.1 OR 2.2	1.1 OR 2.4	1.1 OR 2.6
Y Bound	1.2 OR 2.2	1.2 OR 2.4 OR 3.2	1.2 OR 2.6
Below	1.3 OR 2.2	1.3 OR 2.4	1.3 OR 2.6

SOLUTION:

- Error handling makes string like #DIV/0!